

Robot Localization by Hidden Markov Model

Introduction:

In this step-by-step guide, we will walk through a concrete example of robot localization using Hidden Markov Models (HMMs) in a 3x3 grid. We will see how to update and normalize beliefs about the robot's position based on sensor readings over three iterations. This approach helps refine the robot's location probabilities in the presence of noisy sensor data. Note that this document is generated through interaction with GPT-4o.

Grid States

Consider a simple 3x3 grid, where each cell represents a possible state S of the robot.

```
S1 S2 S3
S4 S5 S6
S7 S8 S9
```

Emission Probabilities

The robot has sensors that provide noisy distance readings to the nearest walls. Possible observations O are 'Near' (N) or 'Far' (F).

- **For state S5 (center):**
 - $P(O = N \mid S5) = 0.7$
 - $P(O = F \mid S5) = 0.3$
- **For corner states (S1, S3, S7, S9):**
 - $P(O = N \mid S1) = 0.9$
 - $P(O = F \mid S1) = 0.1$
- **For side states (S2, S4, S6, S8):**
 - $P(O = N \mid S2) = 0.8$
 - $P(O = F \mid S2) = 0.2$

Initial Belief State

Start with a uniform distribution, assuming the robot is equally likely to be in any state.

```
Belief = [1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 1/9]
```

Transition Model

The robot can move up, down, left, or right with equal probability.

Examples:

- **From S5:**
 - Up (S2), Down (S8), Left (S4), Right (S6) each with probability 1/4.
 - **From S1 (corner):**
 - Right (S2), Down (S4) each with probability 1/2.
 - **From S4 (side):**
 - Right (S5), Up (S1), Down (S7) each with probability 1/3.
-

Example 1: Robot Without Movement

In this example, the robot's belief state is updated based on three sensor readings ('Near', 'Near', 'Far') with no movements.

Iteration #1: Sensor Reading "Near"

1. **Update Belief for All States** using the sensor model:

```
Updated_Belief[S1] = P(0=N | S1) * Belief[S1] = 0.9 * (1/9) = 0.1
Updated_Belief[S2] = P(0=N | S2) * Belief[S2] = 0.8 * (1/9) = 0.0889
Updated_Belief[S3] = P(0=N | S3) * Belief[S3] = 0.9 * (1/9) = 0.1
Updated_Belief[S4] = P(0=N | S4) * Belief[S4] = 0.8 * (1/9) = 0.0889
Updated_Belief[S5] = P(0=N | S5) * Belief[S5] = 0.7 * (1/9) = 0.0778
Updated_Belief[S6] = P(0=N | S6) * Belief[S6] = 0.8 * (1/9) = 0.0889
Updated_Belief[S7] = P(0=N | S7) * Belief[S7] = 0.9 * (1/9) = 0.1
Updated_Belief[S8] = P(0=N | S8) * Belief[S8] = 0.8 * (1/9) = 0.0889
Updated_Belief[S9] = P(0=N | S9) * Belief[S9] = 0.9 * (1/9) = 0.1
```

2. **Normalize Beliefs:**

```
Sum = 0.1 + 0.0889 + 0.1 + 0.0889 + 0.0778 + 0.0889 + 0.1 + 0.0889 + 0.1 =
0.8334
```

```
Normalized_Belief[S1] = 0.1 / 0.8334 ≈ 0.12
Normalized_Belief[S2] = 0.0889 / 0.8334 ≈ 0.1067
Normalized_Belief[S3] = 0.1 / 0.8334 ≈ 0.12
Normalized_Belief[S4] = 0.0889 / 0.8334 ≈ 0.1067
Normalized_Belief[S5] = 0.0778 / 0.8334 ≈ 0.0934
```

```
Normalized_Belief[S6] = 0.0889 / 0.8334 ≈ 0.1067
Normalized_Belief[S7] = 0.1 / 0.8334 ≈ 0.12
Normalized_Belief[S8] = 0.0889 / 0.8334 ≈ 0.1067
Normalized_Belief[S9] = 0.1 / 0.8334 ≈ 0.12
```

The normalized belief state after the first iteration:

```
Belief = [0.12, 0.107, 0.12, 0.107, 0.093, 0.107, 0.12, 0.107, 0.12]
```

Iteration #2: Sensor Reading "Near"

1. Update Belief for All States:

```
Updated_Belief[S1] = P(0=N | S1) * Belief[S1] = 0.9 * 0.12 ≈ 0.108
Updated_Belief[S2] = P(0=N | S2) * Belief[S2] = 0.8 * 0.107 ≈ 0.0856
Updated_Belief[S3] = P(0=N | S3) * Belief[S3] = 0.9 * 0.12 ≈ 0.108
Updated_Belief[S4] = P(0=N | S4) * Belief[S4] = 0.8 * 0.107 ≈ 0.0856
Updated_Belief[S5] = P(0=N | S5) * Belief[S5] = 0.7 * 0.093 ≈ 0.0651
Updated_Belief[S6] = P(0=N | S6) * Belief[S6] = 0.8 * 0.107 ≈ 0.0856
Updated_Belief[S7] = P(0=N | S7) * Belief[S7] = 0.9 * 0.12 ≈ 0.108
Updated_Belief[S8] = P(0=N | S8) * Belief[S8] = 0.8 * 0.107 ≈ 0.0856
Updated_Belief[S9] = P(0=N | S9) * Belief[S9] = 0.9 * 0.12 ≈ 0.108
```

2. Normalize Beliefs:

```
Sum = 0.108 + 0.0856 + 0.108 + 0.0856 + 0.0651 + 0.0856 + 0.108 + 0.0856 +
0.108 = 0.8385
```

```
Normalized_Belief[S1] = 0.108 / 0.8385 ≈ 0.129
Normalized_Belief[S2] = 0.0856 / 0.8385 ≈ 0.102
Normalized_Belief[S3] = 0.108 / 0.8385 ≈ 0.129
Normalized_Belief[S4] = 0.0856 / 0.8385 ≈ 0.102
Normalized_Belief[S5] = 0.0651 / 0.8385 ≈ 0.078
Normalized_Belief[S6] = 0.0856 / 0.8385 ≈ 0.102
Normalized_Belief[S7] = 0.108 / 0.8385 ≈ 0.129
Normalized_Belief[S8] = 0.0856 / 0.8385 ≈ 0.102
Normalized_Belief[S9] = 0.108 / 0.8385 ≈ 0.129
```

The normalized belief state after the second iteration:

```
Belief = [0.129, 0.102, 0.129, 0.102, 0.078, 0.102, 0.129, 0.102, 0.129]
```

Iteration #3: Sensor Reading "Far"

1. Update Belief for All States:

```
Updated_Belief[S1] = P(O=F | S1) * Belief[S1] = 0.1 * 0.129 ≈ 0.0129
Updated_Belief[S2] = P(O=F | S2) * Belief[S2] = 0.2 * 0.102 ≈ 0.0204
Updated_Belief[S3] = P(O=F | S3) * Belief[S3] = 0.1 * 0.129 ≈ 0.0129
Updated_Belief[S4] = P(O=F | S4) * Belief[S4] = 0.2 * 0.102 ≈ 0.0204
Updated_Belief[S5] = P(O=F | S5) * Belief[S5] = 0.3 * 0.078 ≈ 0.0234
Updated_Belief[S6] = P(O=F | S6) * Belief[S6] = 0.2 * 0.102 ≈ 0.0204
Updated_Belief[S7] = P(O=F | S7) * Belief[S7] = 0.1 * 0.129 ≈ 0.0129
Updated_Belief[S8] = P(O=F | S8) * Belief[S8] = 0.2 * 0.102 ≈ 0.0204
Updated_Belief[S9] = P(O=F | S9) * Belief[S9] = 0.1 * 0.129 ≈ 0.0129
```

2. Normalize Beliefs:

```
Sum = 0.0129 + 0.0204 + 0.0129 + 0.0204 + 0.0234 + 0.0204 + 0.0129 +
0.0204 + 0.0129 = 0.157
```

```
Normalized_Belief[S1] = 0.0129 / 0.157 ≈ 0.0822
Normalized_Belief[S2] = 0.0204 / 0.157 ≈ 0.130
Normalized_Belief[S3] = 0.0129 / 0.157 ≈ 0.0822
Normalized_Belief[S4] = 0.0204 / 0.157 ≈ 0.130
Normalized_Belief[S5] = 0.0234 / 0.157 ≈ 0.149
Normalized_Belief[S6] = 0.0204 / 0.157 ≈ 0.130
Normalized_Belief[S7] = 0.0129 / 0.157 ≈ 0.0822
Normalized_Belief[S8] = 0.0204 / 0.157 ≈ 0.130
Normalized_Belief[S9] = 0.0129 / 0.157 ≈ 0.0822
```

The normalized belief state after the third iteration:

```
Belief = [0.0822, 0.130, 0.0822, 0.130, 0.149, 0.130, 0.0822, 0.130,
0.0822]
```

Example 2: Robot With Movement

In this example, we use a **3x3 grid** where the robot updates its belief based on movements and sensor readings.

Iteration 1: Sensor Reading "Near" Without Movement

Similar to Example 1 after normalization:

```
Belief = [0.12, 0.107, 0.12, 0.107, 0.093, 0.107, 0.12, 0.107, 0.12]
```

Iteration 2: Robot Moves Up, then Reads Sensor "Near"

Prediction Step (Moving Up)

Each state receives contributions from the state directly below it:

1. New Belief after Moving Up:

- **Corner States:**
 - (S1) receives contribution from (S4): ($\text{New_Belief}[S1] = \text{Belief}[S4] = 0.107$)
 - (S3) receives contribution from (S6): ($\text{New_Belief}[S3] = \text{Belief}[S6] = 0.107$)
 - (S7): No contribution, as there is no cell below it $\rightarrow$ ($\text{New_Belief}[S7] = 0$)
 - (S9): No contribution, as there is no cell below it $\rightarrow$ ($\text{New_Belief}[S9] = 0$)
- **Side States:**
 - (S2) receives contribution from (S5): ($\text{New_Belief}[S2] = \text{Belief}[S5] = 0.093$)
 - (S4) receives contribution from (S7): ($\text{New_Belief}[S4] = \text{Belief}[S7] = 0.12$)
 - (S6) receives contribution from (S9): ($\text{New_Belief}[S6] = \text{Belief}[S9] = 0.12$)
 - (S8): No contribution, as there is no cell below it $\rightarrow$ ($\text{New_Belief}[S8] = 0$)
- **Center State:**
 - (S5) receives contribution from (S8): ($\text{New_Belief}[S5] = \text{Belief}[S8] = 0.107$)

Resulting `New_Belief` after Moving Up:

```
New_Belief = [0.107, 0.093, 0.107, 0.12, 0.107, 0.12, 0, 0, 0]
```

Sensor Update for Reading "Near"

After predicting the new beliefs, we apply the sensor reading "Near":

2. Updated Belief based on Sensor "Near":

```
Updated_Belief[S1] = 0.9 * New_Belief[S1] = 0.9 * 0.107 ≈ 0.0963
Updated_Belief[S2] = 0.8 * New_Belief[S2] = 0.8 * 0.093 ≈ 0.0744
Updated_Belief[S3] = 0.9 * New_Belief[S3] = 0.9 * 0.107 ≈ 0.0963
Updated_Belief[S4] = 0.8 * New_Belief[S4] = 0.8 * 0.12 ≈ 0.096
Updated_Belief[S5] = 0.7 * New_Belief[S5] = 0.7 * 0.107 ≈ 0.0749
Updated_Belief[S6] = 0.8 * New_Belief[S6] = 0.8 * 0.12 ≈ 0.096
Updated_Belief[S7] = 0.9 * New_Belief[S7] = 0.9 * 0 ≈ 0
Updated_Belief[S8] = 0.8 * New_Belief[S8] = 0.8 * 0 ≈ 0
Updated_Belief[S9] = 0.9 * New_Belief[S9] = 0.9 * 0 ≈ 0
```

3. Normalization:

```
Sum = 0.0963 + 0.0744 + 0.0963 + 0.096 + 0.0749 + 0.096 + 0 + 0 + 0 =
0.5339
```

```
Normalized_Belief = [0.1804, 0.1394, 0.1804, 0.1799, 0.1403, 0.1799,
0, 0, 0]
```

Final Belief after Iteration 2:

```
Belief = [0.1804, 0.1394, 0.1804, 0.1799, 0.1403, 0.1799, 0, 0, 0]
```

Iteration 3: Robot Moves Right, then Reads Sensor "Far"

Prediction Step (Moving Right)

Each state receives contributions **from the state directly to the left of it**. Let's compute the `New_Belief` values accordingly.

1. New Belief after Moving Right:

- **Corner States:**

- (S1): No contributions (there's no cell to the left) $\rightarrow (\text{New_Belief}[S1] = 0)$
- (S3) receives from (S2): $(\text{New_Belief}[S3] = \text{Belief}[S2] \approx 0.1394)$
- (S7): No contributions (there's no cell to the left) $\rightarrow (\text{New_Belief}[S7] = 0)$
- (S9) receives from (S8): $(\text{New_Belief}[S9] = \text{Belief}[S8] = 0)$

- **Side States:**

- (S2) receives from (S1): ($\text{New_Belief}[S2] = \text{Belief}[S1] = 0.1804$)
- (S4): No contributions (there's no cell to the left) $\rightarrow$ ($\text{New_Belief}[S4] = 0$)
- (S6) receives from (S5): ($\text{New_Belief}[S6] = \text{Belief}[S5] \approx 0.1403$)
- (S8) receives from (S7): ($\text{New_Belief}[S8] = \text{Belief}[S7] = 0$)
- **Center State (S5):**
 - (S5) receives contribution from (S4): ($\text{New_Belief}[S5] = \text{Belief}[S4] \approx 0.1799$)

Resulting `New_Belief` after Moving Right:

```
New_Belief = [0, 0.1804, 0.1394, 0, 0.1799, 0.1403, 0, 0, 0]
```

Sensor Update for Reading "Far"

After the prediction step, we update based on the sensor reading "Far." The emission probabilities for "Far" are:

- **Corner States:** 0.1
- **Side States:** 0.2
- **Center State (S5):** 0.3

Using these probabilities:

1. Updated Belief based on Sensor "Far":

```
Updated_Belief[S1] = 0.1 * New_Belief[S1] = 0.1 * 0 = 0
Updated_Belief[S2] = 0.2 * New_Belief[S2] ≈ 0.2 * 0.1804 ≈ 0.03608
Updated_Belief[S3] = 0.1 * New_Belief[S3] ≈ 0.1 * 0.1394 ≈ 0.01394
Updated_Belief[S4] = 0.2 * New_Belief[S4] = 0.2 * 0 = 0
Updated_Belief[S5] = 0.3 * New_Belief[S5] ≈ 0.3 * 0.1799 ≈ 0.05397
Updated_Belief[S6] = 0.2 * New_Belief[S6] ≈ 0.2 * 0.1403 ≈ 0.02806
Updated_Belief[S7] = 0.1 * New_Belief[S7] = 0.1 * 0 = 0
Updated_Belief[S8] = 0.2 * New_Belief[S8] = 0.2 * 0 = 0
Updated_Belief[S9] = 0.1 * New_Belief[S9] = 0.1 * 0 = 0
```

2. Normalization:

```
Sum = 0 + 0.03608 + 0.01394 + 0 + 0.05397 + 0.02806 + 0 + 0 + 0 =
0.13205
```

```
Normalized_Belief ≈ [0, 0.2731, 0.1056, 0, 0.4085, 0.2128, 0, 0, 0]
```

Final Belief after Iteration 3:

```
Belief ≈ [0, 0.2731, 0.1056, 0, 0.4085, 0.2128, 0, 0, 0]
```

Summary

Through each iteration, the belief state is updated and normalized based on the sensor readings ('Near', 'Near', 'Far'). The robot's location becomes more probabilistically defined with each subsequent reading. By using the Hidden Markov Model, we efficiently account for transitions and noisy sensor data, refining our understanding of the robot's position in the grid.